

Negative Sentiment Causal Effect on Code Contribution in Open-Source Communities



github.com/sailuh

Adam Graham, Samantha Limon, Phuong Pham, Jared Seto
Sponsors: Carlos Paradis and Rick Kazman, Software Analytics Insight Lab
ICS 496 Capstone Project - Spring 2026



What is SAILUH?

Software Analytics Insight Lab at the University of Hawaii at Mānoa.

- GitHub organization for a UH Mānoa research lab that builds tools to analyze software projects and developer communities.

Objectives

Our overall objective was to build an end-to-end pipeline that integrates 4 components to gain insight on the causal effect of negative sentiment on code contribution in open-source communities. The goals were to (1) extend a sentiment classifier to label comments according to their sentiment, (2) take those labels and provide an aggregated engagement sentiment score for a developer, (3) use the aggregated sentiment as weights for a graph representation of a developer's engagement network, and (4) integrate these sentiment variables into a causal analysis notebook to understand the cause and effect of a developer's negative sentiment on overall code contribution for an open-source project.

Methodology

Project Management

- Agile software development
- Documented GitHub Issues/PRs/Project Board

Roles

- **Project Manager:** Adam
- **Liaison:** Samantha
- **Quality Manager:** Phuong
- **Architect:** Jared

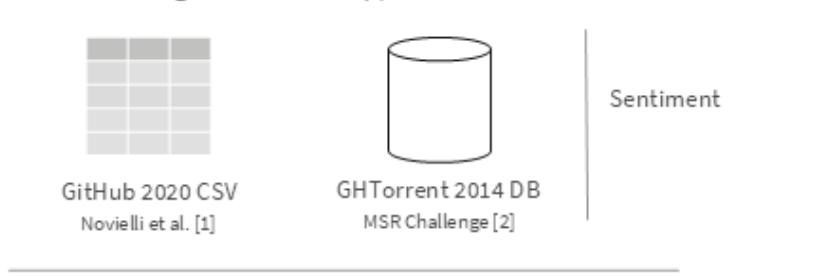
Sentiment and Emotion Datasets : : CHEAT SHEET

About

Manually annotated sentiment/emotion labels can be useful for training and evaluating novel machine learning classifiers. However, these labels lack sufficient metadata to be extended with new observations for the same projects that were created. This cheatsheet demonstrates the reconstruction process used to expand labels' metadata, define reproducible project configuration files, and make them usable in Kaialu for new analyses.

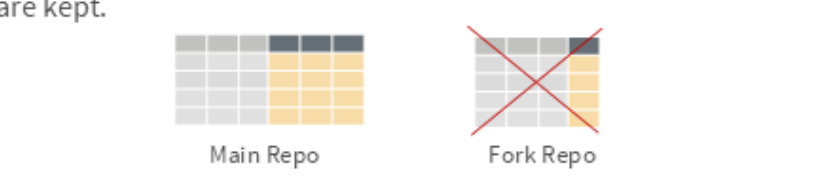
Raw Dataset

The following must be set up/downloaded:



Label Selection Criteria

Sentiment labels span both main project repos and their forks. However, since forked repos contain a small fraction (< 10%) of labeled comments, only comments from the project's main repo are kept.



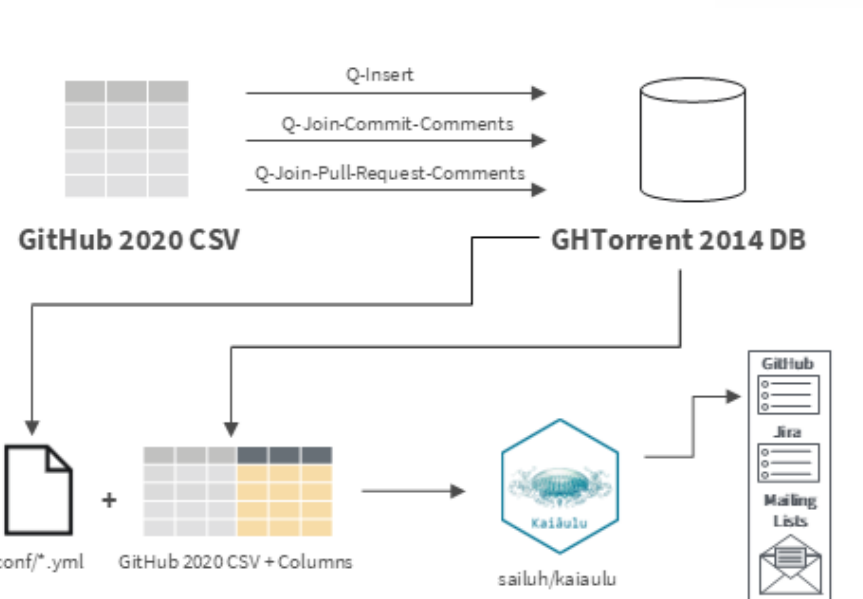
The Jira Emotion CSV contains emotion labels manually determined from multiple raters. Only comments with labels that show higher inter-rater agreement are kept.



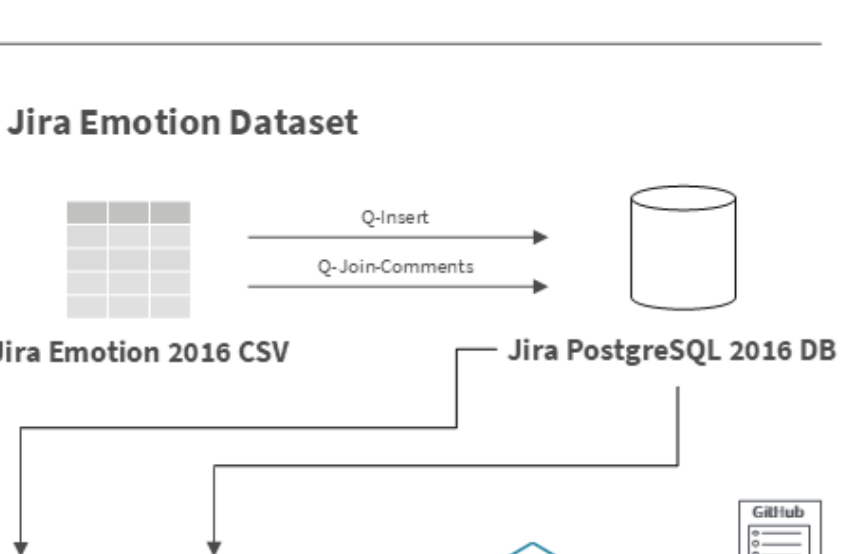
Kaialu

Reproducing Datasets with Kaialu

GitHub Sentiment Dataset



Jira Emotion Dataset



Kaialu

Related Notebooks

See supplementary notebooks that go in-depth on the documentation on making the Gold Standard and Jira Emotion datasets compatible with Kaialu.

- 1. `load_sentiment_csv_to_mysql.ipynb`
- 2. `explore_relevant_projects.ipynb`
- 3. `scale_config_files.ipynb`
- 4. `add_sentiment_to_kaialu.ipynb`

sailuh/emotion_jira_dataset

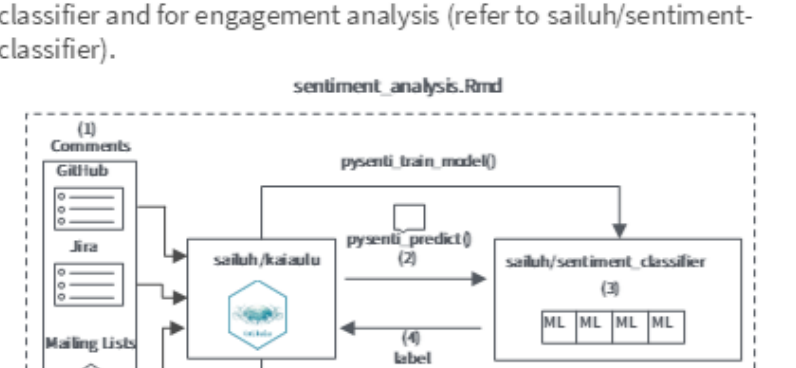
- 1. `emotion_csv_inter_rater_agreement.ipynb`

Data Expansion

Through this pipeline, we recover sufficient metadata to re-obtain the data using Kaialu. This metadata enables two types of dataset expansion using Kaialu.

- 1. **Temporal expansion:** Re-download the same projects' comments from 2014 through today.
- 2. **Horizontal expansion:** Download additional data sources for those same projects (e.g., source code, version control history) that can be linked back to sentiment labels.

The labeled comments can be used to train the sentiment classifier and for engagement analysis (refer to sailuh/sentiment-classifier).



Kaialu

Sentiment Classifier : : CHEAT SHEET

About

Kaialu provides support for the download of comment data from multiple sources (e.g. GitHub, Jira, and mailing lists) (see refresher-cheatsheet.pdf). This cheatsheet covers how to add sentiment labels to each comment by using machine learning classifiers for sentiment analysis.

Project Config Setup

Before running a vignette, you must set up your project configuration file (examples in conf/) to specify the paths to download the data.

Required Fields

- `issue_tracker:`
 - github:
 - project_key_1:
 - owner:
 - repo:
 - issue_or_pr_comment:
 - issue_event:
 - comment:
 - comment_comments:
 - pr_comments:
- `jira:`
 - project_key_1:
 - issue_comments:
 - mod_index:
 - mailing_list:
 - project_key_1:
 - papermail:
 - project_key_1:

You should also specify the required external tools in tools.yml:

Tools Config Setup

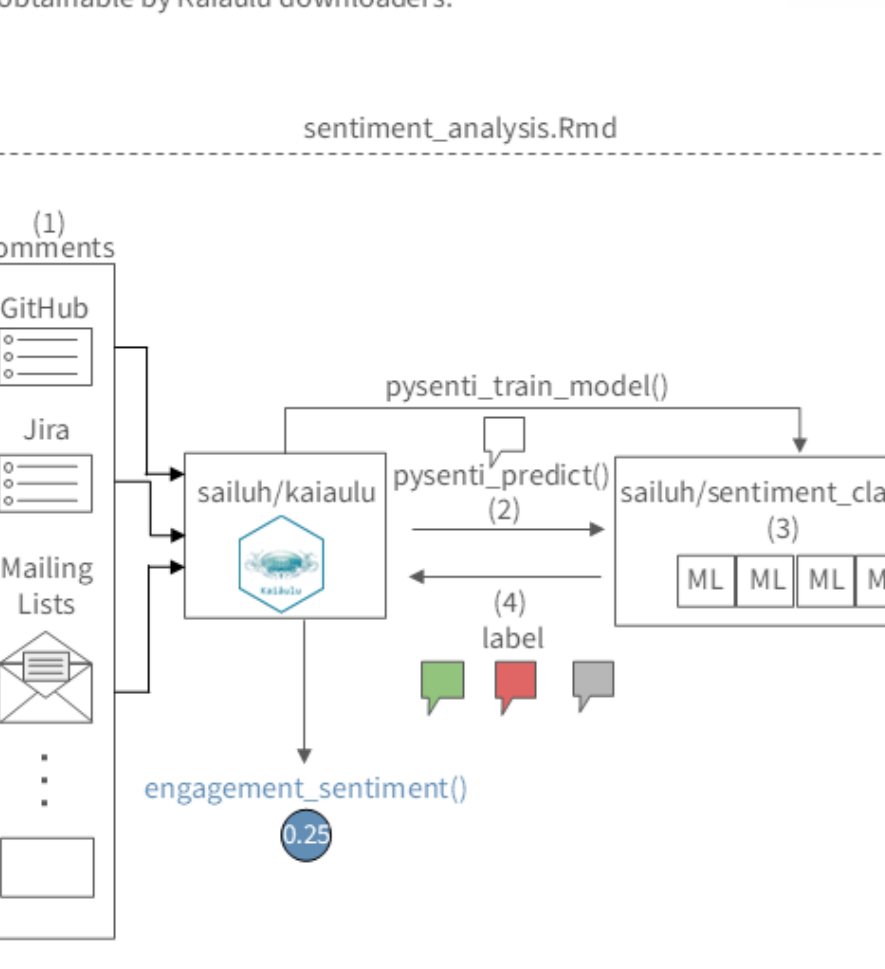
Required Fields

- `psysnt:`
 - 1. `sentiment_classifier/exec/train_or_predict.py`
 - `perceval:`

Kaialu

Comment Sentiment

Kaialu's `sentiment_analysis.Rmd` interfaces with a family of sentiment classifier models. The classifier leverages comment data (1) obtainable by Kaialu downloaders.



Kaialu passes comments (1) to the sentiment classifier using `psysnt_predict()` (2). The tools.yml path for the sentiment classifier follows the same pattern as the other third-party tools.

The sentiment classifier labels each comment as **positive** (green), **negative** (red), or **neutral** (grey) using machine learning models (3) and returns that label (4) to Kaialu.

The sentiment labels can also be used as a measure of project emotional engagement.

Kaialu

Related Vignettes

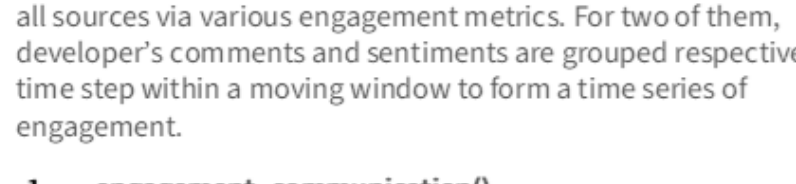
See supplementary notebooks that go in-depth on downloading and parsing communication data and conducting the full sentiment analysis workflow.

- 1. `download_github_events.Rmd`
- 2. `download_github_pull_requests.Rmd`
- 3. `sentiment_analysis.Rmd`

Engagement Metrics

Kaialu supports time-series analysis of developer comments across all sources via various engagement metrics. For two of them, developer's comments and sentiments are grouped respectively by time step within a moving window to form a time series of engagement.

- 1. `engagement_communication()`
 - Counts the overall number of messages a developer sent.
- 2. `engagement_sentiment()`
 - Produces an engagement sentiment score.



Kaialu

Network Transform: : CHEAT SHEET

About

Kaialu unifies a variety of data sources (e.g. git logs, mailing lists) in a consistent object-oriented graph structure using S3 (R's object-oriented system). This abstraction allows future graph functions to be de-coupled from the data source. Graph functions will also apply the correct transformations depending on the graph object being passed as parameter (similarly to a print() function).

Downloaders

Kaialu downloaders allow different raw data to be obtained. For reference, see refresher-cheatsheet:cheatsheet:refresher-cheatsheet.pdf

- 1. `download_jira_issues()`
 - Downloads JIRA data locally as JSON files.
 - 2. ...

Parsers

Parse functions format raw data into structured tables that are reusable across multiple transforms.

- 1. `parse_gitlog()`
 - Parses a git repository into a commit table.
 - 2. `parse_jira_replies()`
 - Parses Jira issue comments into a reply table.
 - 3. ...

Transformers

Transform functions construct S3 graph objects from parsed tables by defining nodes, edges, and relationships.

- 1. `transform_gitlog_to_bipartite_network()`
 - Git Log commit table -> bipartite graph.
 - 2. `transform_gitlog_to_multimodal_network()`
 - Git Log commit table -> multimodal graph.
 - 3. `transform_reply_to_bipartite_network()`
 - Reply table -> person-issue bipartite graph.
 - 4. ...

Kaialu

Network Transform Pipeline

Transforms utilize a graph structure which generalizes different types of graphs, regardless of their data source.

Graph Constructors

- 1. `model_unimodal_graph()`
 - Represents a unimodal graph.
- 2. `model_multimodal_graph()`
 - Represents a multimodal graph.

S3 Functions

S3 methods execute the correct behavior based on the graph's class, allowing for analyses functions to work regardless of the data's origin.

- Examples of S3 Generic Functions:
- 1. `bipartite_graph_projection()`
 - Performs bipartite projection on multimodal graphs.
- 2. `temporal_graph_projection()`
 - Performs temporal projection on multimodal graphs.
- 3. `community_oslom()`
 - Runs OSLOM community detection on a graph.
- 4. `graph_to_dcmj()`
 - Exports graph to a .dcmj file.

Example of S3 Auxiliary Functions:

- 5. `subset_bipartite_from_multimodal()`
 - Slices a bipartite graph from a multimodal graph.
- 6. `get_bidirected_edges()`
 - Returns all mutual edge pairs from a graph.

Custom Graph Weights

Any numeric value — such as sentiment scores or custom weight schemes — can be assigned as edge **weights** without modifying any existing transforms.

- 1. `transform_reply_to_bipartite_network(weight, ...)`
- 2. `transform_gitlog_to_bipartite_network(weight, ...)`
- 3. ...

Kaialu

Pykumu : : CHEAT SHEET

About

Pykumu is a Python package that serves as a wrapper around the Tetrad Java API (via JPyPee), providing an interface for causal analysis algorithms. Pykumu is a fork of py-tetrad/pytetrad.

Pykumu API

Pykumu wraps the Tetrad Java API through JPyPee, exposing individual modules in pykumu.api (data, score, algorithm, graph, knowledge, bootstrapping) that provide direct function level control over each step of the causal search pipeline. Pykumu expands on the TetradSearch API by breaking its single object, shared state pipeline into individual modules.

Command Line Interface

Exec scripts provides command line access to Pykumu API package, allowing its causal analysis capabilities to be accessed via the terminal or server side without writing Python code.

Kumu

Causal Analysis Workflow

- 1. `sailuh/kaialu`
- 2. `Dataset`
- 3. `Time Lag`
- 4. `Indicator Features`
- 5. `Null Features`
- 6. `Causal Search`
- 7.1 `1-PNEF Threshold`
- 7.2 `Domain Knowledge`
- 8. `Domain Knowledge Causal Search`

API

From an example of a dataset, see `issue_social_smells_showcase.Rmd`. Refer to cheatsheets/social-smells-cheatsheet.pdf

tetrad.py

After applying time lag and features, jpypee is imported to utilize API and turn the dataset into a Tetrad compatible format.

- 1. `tetrad_jvm_start()`
 - Import jpypee to start JVM and load Tetrad jar file.
- 2. `transform_pandasdf_to_tetrad_boxdataset()`
 - Convert pandas DataFrame into Tetrad BoxDataset and initialize search state.

causal.py

Causal search is run on dataset including null features, used to derive 1-PNEF.

- 1. `score_sem_bic()`
 - Configure SEM BIC score.
- 2. `bootstrapping()`
 - Configure bootstrapping parameters.
- 3. `algorithm_fgcs()`
 - Performs Fast Greedy Equivalence Search (FGES) algorithm.
- 4. `algorithm_boss()`
 - Performs Best Order Score Search (BOSS) algorithm.

knowledge.py

Causal search is ran again with domain knowledge applied.

- 1. `parse_knowledge_txt()`
 - Load forbidden/required edges and temporal tiers from a .txt file into Tetrad knowledge object.

graph.py

Causal graph output is converted and saved in JSON format that can be loaded and viewed in Tetrad GUI. Parsing graph data into nodes, edgeset, and edge_type_probabilities allows 1-PNEF to be applied.

- 1. `transform_graph_java_to_graph_json()`
 - Convert Tetrad graph object to JSON format.
- 2. `parse_graph_json()`
 - Parse Tetrad JSON data into nodes, edgeset, and edge_type_probabilities datasets.

Kumu

Challenges

- Establishing an understanding of the project scope and documentation requirements.
- All team members have limited prior experience with the R programming language.
- Maintaining a consistent system architecture when implementing features within the Kaialu repository to align with predefined workflows and design patterns.

Learnings and Conclusion

We learned how to contribute to open-source software by expanding the SAILUH ecosystem and discovered how to effectively extend SAILUH's capabilities through mutually agreed upon specifications from our sponsors. We were able to extend these capabilities with an improved sentiment classifier, new developer engagement metrics, an improved graph model, and causal analysis, ensuring we write maintainable code for future contributors. The project improved our technical skills by working on a large-scale software project, utilizing industry standard developer tools like RStudio, GitHub, and Git. The future consideration for this project is to completely bridge our 4 components as a unified end-to-end pipeline.

Tasks Accomplished

- **Sentiment Classifier:** Built an end-to-end pipeline that produces sentiment-labeled communication comments compatible with Kaialu.
- **New Metrics:** Implemented new metrics that measure engagement, productivity, sentiment, and communication among developers in a project.
- **Network Refactoring:** Introduced a standardized S3 graph model with typed constructors and dispatch methods for extensible downstream analysis.
- **Causal Analysis:** Implemented Pykumu API package, a Python interface to Tetrad's causal analysis library.

Technologies

- **Developer Tools:** RStudio, Jupyter Notebook, Visual Studio Code, GitHub, Git
- **Languages:** R, Python
- **Third-Party Tools:** Perceval, Depends, Understand, Jira, SCC, OSLOM, DV8, Sentiment Classifiers, PyTetrad